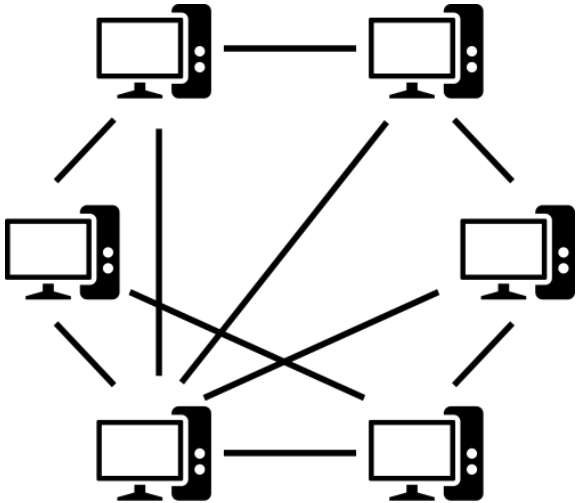
Evaluating Scalability and Latency in Distributed Hash Tables

Daryl Ho, Josh Kindarara

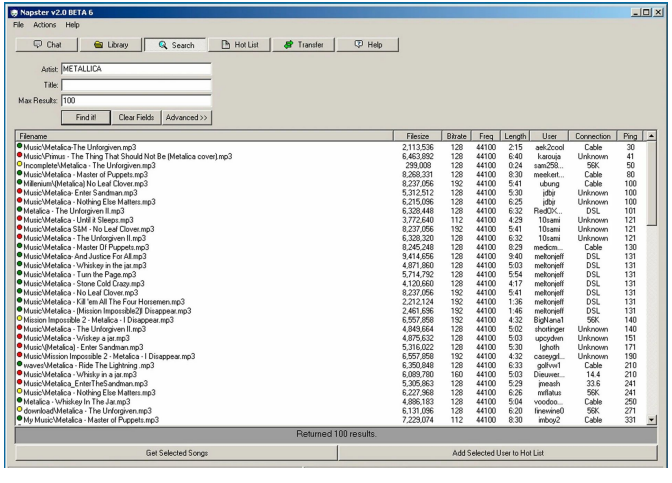
Introduction to P2P Systems

- Goal: We want distributed systems to scale based on the number of nodes (horizontal scalability)
- Concept: Peer-to-Peer System
  - A distributed application architecture that partitions tasks or workloads between peers



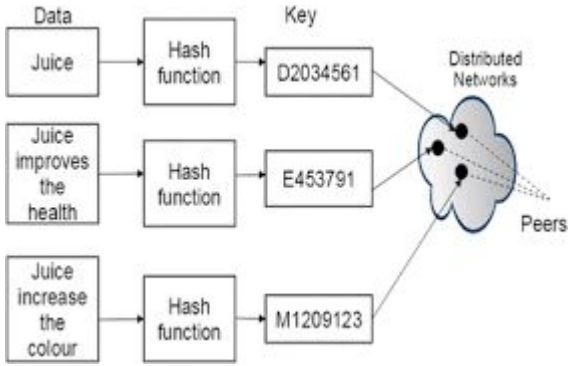
Related Work

- **Napster**
  - Distributed file system - clients connected to server hold files
  - Server holds hash table that stores the location of files on client machines
- Main Problem with Napster
  - Central server = central point of failure and source of congestion
  - For an efficient distributed system with better performance, need to decentralize



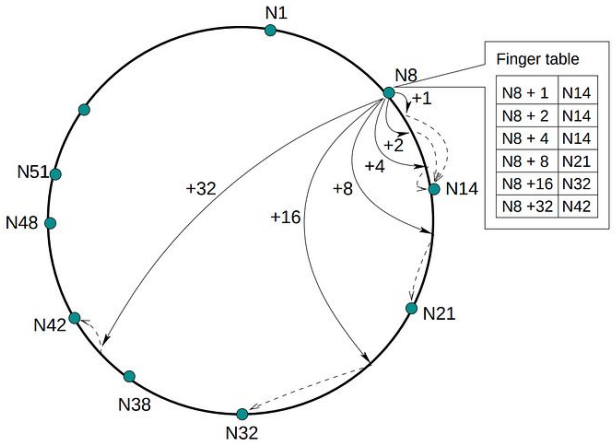
Distributed Hash Tables (DHTs)

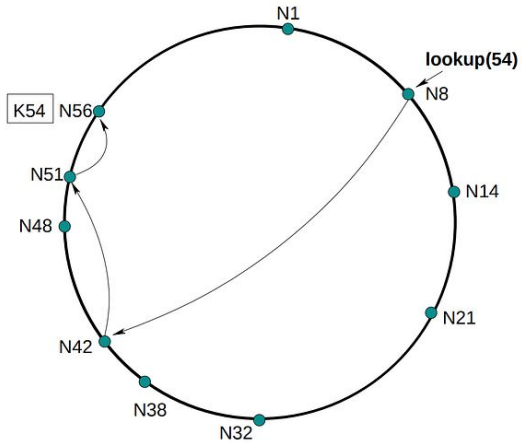
- A data structure that allows the insertion, lookup, and deletion of objects in a distributed setting
- Issue: Expensive to store and update location of ALL nodes and objects at ALL nodes
- Problem: How to efficiently perform lookups
- Solution: Each node only stores location of some nodes to reduce latency and message cost of routing (lookups/inserts)



Chord DHT

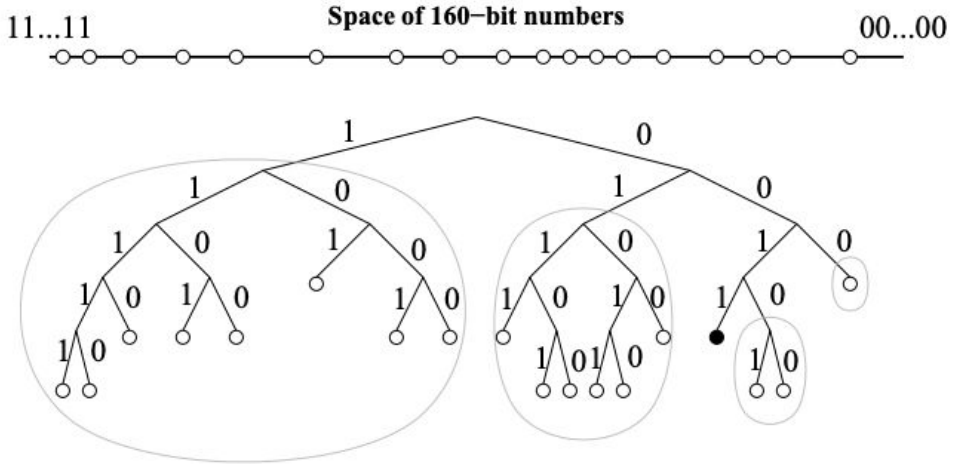
- Nodes are structured in a ring formation where each node has successors and predecessors they are connected to
- Nodes use a “finger table” to store the locations of their closest successors
- Lookups consist of step by step jumps through nodes until the designated node or item is found





Kademlia DHT

- Nodes are contained in buckets where each node's distance to others is the XOR of the nodes IDs
- Nodes use a "history table" to store the locations of the closest nodes
- Lookups consist of multicast messages to nodes until the designated node or item is found



Design

- Main Performance Concerns:
  - Load balancing
  - Fault-tolerance
  - Efficiency of lookups and inserts
  - Locality
- Design Solutions
  - Consistent Hashing
  - Replication
  - Stabilization
  - Timeout-Based RPC and Retry Design

Consistent Hashing

- We compute the identifier by using C++'s std::hash (compiler-dependent hashing function)
- Hash the socket address of a node and mod it by 2^m
- Consistent hashing minimizes data movement when nodes join/leave which increases **scalability**
- **Load Balancing** - keys are evenly distributed
- Keeps the whole system **decentralized**, so there's no central coordinator
    - In Chord, nodes can join through any node in the ring
    - In Kademlia, nodes can join through any open bucket

Replication

- We replicated data on r immediate successors
- Choosing immediate successors rather than random nodes increases **availability** and **locality**
  - Consecutive nodes would need to fail for data loss, which is not likely
- Eventual **consistency**
  - Update owner first
  - Asynchronously push updates to replicas
  - Our PUT RPC eventually reaches all r replicas
- Since we have replication, our GET does a local data check first before forwarding a request to a node in its finger table

Stabilization

- Each node runs an internal stabilizer thread
  - Runs every few seconds
  - Finger tables are fixed, predecessors and successors are checked, etc.,
- Ensures **eventual consistency**
- Runs in a timeout-based RPC layer, so stabilization doesn’t block
  - Allowing **concurrency**

Timeout-Based RPC and Retry Design

- Our RPC sends/receives have a 1-second timeout
- Will automatically retry on failure a maximum of 3 times
- Our RPC accept loop is non-blocking
  - We’re not infinitely waiting for dead nodes
- Timeouts allow
  - Fast detection of failed peer nodes
  - **Concurrency** without deadlocks and long hang times
  - **Fault isolation** - one slow or failing node doesn’t stall the whole ring

Architecture Overview: Chord

- A Node class is initialized with the following
  - u64 ID
  - IP Address
  - Port Number
- Nodes joining are assigned a predecessor and successor as well as a finger table
- Each finger table is initialized with m = 8 entries
- When a lookup is called on a node
  - The node checks if they have the item
  - If it doesn't have it, then it refers to the finger table to find the largest successor node that could potentially have the item
  - If no such node exists, it sends it to its successor node

Architecture Overview: Kademlia

- A Node class is initialized with the following
  - u64 ID
  - IP Address
  - Port Number
- Nodes joining are added to an open bucket and assigned a routing table
- Each history table is initialized with m = 160 k-buckets with k = 20 contacts
- When a lookup is called on a node
  - The node checks if they have the item
  - If it doesn't have it, then it refers to the history table to find and repeatedly query the $\alpha = 3$ closest unqueried nodes sorted by XOR distance in parallel for the lookup until no more nodes remain

Demo

Future Plans

- Garbage Collection
  - Without it, nodes can accumulate garbage
  - Inconsistency increases as nodes join and leave

Summary

- Chord and Kademlia both solve the distributed lookup problem using routing tables and O(log n) lookup procedures, but take very different approaches.
  - Chord performs sequential, ring-based routing using finger tables.
  - Kademlia performs parallel, XOR-distance-based searches using k-buckets.
- Kademlia is generally more fault-tolerant and faster in high-latency environments, while Chord's structure provides clean theoretical guarantees.
- Our results showed that both systems can be made robust and efficient with sufficient engineering support, but their behavior differs significantly, especially under failures and load.
- Overall, the project gave us practical insight into building scalable, decentralized systems, comparing real-world DHT designs, and showing how scalability and latency are impacted in increasingly large environments.

Thank You